

MQTT 通信案例开发手册



主题	MQTT 通信案例开发手册
文档号	1.0
创建时间	2025-03-18
最后修改	2025-03-18
版本号	1.0
文件名	MQTT 通信案例开发手册 V1.0.pdf
文件格式	Portable Document Format

阅前须知

声明

北京合众恒跃科技有限公司保留随时对其产品进行修正、改进和完善的权利，客户在下单前应获取相关信息的最新版本，并验证这些信息是正确的。本文档一切解释权归北京合众恒跃科技有限公司所有。

简介

北京合众恒跃科技有限公司位于中关村科技园区，公司主要从事嵌入式产品的设计、研发、生产、销售等业务；公司的核心竞争力是对嵌入式产品精准的设计及实现能力，成本控制能力和产品品质保障能力；公司主干研发人员均有多年嵌入式产品开发经验和软硬件技术积累，服务于音视频、图像识别、电力等多个应用领域，公司研发设计的视频转码卡、图像识别卡、电力保护装置等产品在业内享有一定的知名度。

联系方式

北京合众恒跃科技有限公司

电话：010-62129511

邮编：102208

技术支持邮箱：support@hzhytech.com

地址：北京市海淀区安宁庄后街南 1 号 A 区一层 1020 号



官方微信公众号



官方企业微信



合众嵌入式官方店铺



合众 AI 官方店铺

修改记录

版本号	日期	修改人	备注
1.0	2025-03-18	刘瑞萱	初始版本

目录

阅前须知	2
声明	2
简介	2
联系方式	2
修改记录	3
目录	4
第 1 章 MQTT 基础	6
1.1 MQTT 简介	6
1.2 MQTT 基本概念	7
1.3 MQTT 客户端	8
1.4 连接和身份验证	8
1.5 主题订阅和消息发布	9
1.6 断开连接和持久化会话	9
1.7 MQTT 与物联网应用的集成	10
第 2 章 MQTT 客户端库使用	11
2.1 Mosquitto 简介	11
2.2 Mosquitto 的安装	11
2.3 测试用例	11
第 3 章 MQTT 环境搭建	14
3.1 ubuntu 搭建	14
3.2 buildroot 搭建	16
3.2.1 配置选择	16
3.2.2 编译及固化	18
3.3 源码交叉编译	19

3.4 环境测试	26
第 4 章 MQTT 开发案例	27
4.1 案例说明	27
4.2 案例测试	27
4.3 案例编译	28
4.4 关键代码分析	29
第 5 章 常用接口说明	31

第 1 章 MQTT 基础

1.1 MQTT 简介

MQTT 是一种轻量级的消息传递协议，设计用于在低带宽和不稳定网络环境下进行可靠的消息通信。

它是基于发布/订阅模式的，允许设备和应用程序通过发送和接收消息来进行通信，如图 1.1-1 通信方式图。

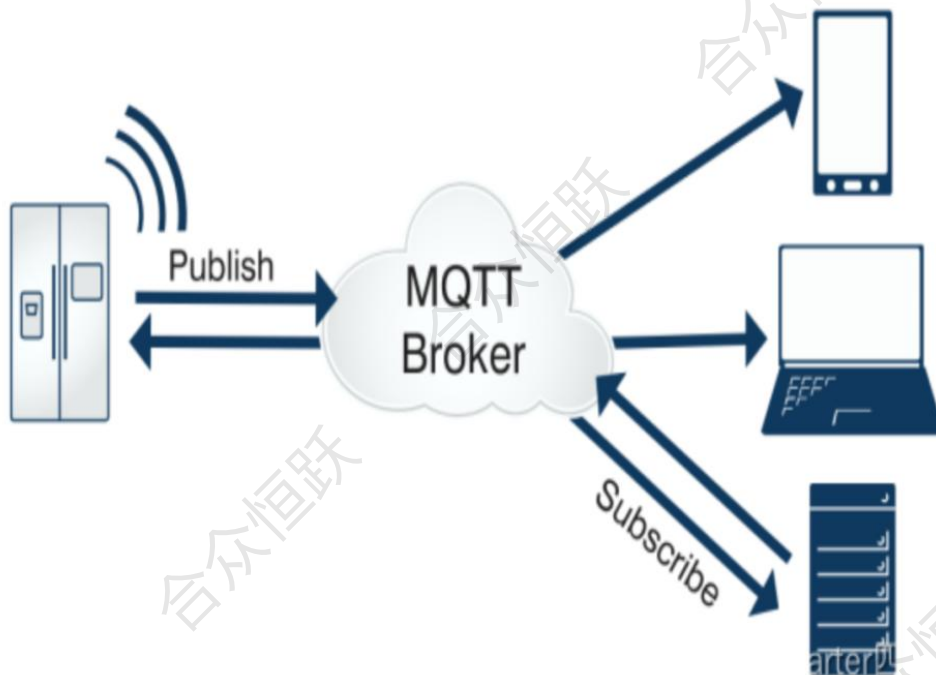


图 1.1-1 通信方式图

MQTT 有如下特点：

- 1) 轻量级：MQTT 协议设计简单，开销小，适合在资源受限的设备和网络上使用。
- 2) 灵活性：支持灵活的主题订阅和消息发布，使得设备和应用程序可以选择性地接收和发送感兴趣的消息。
- 3) 可靠性：提供多个消息传递质量（QoS）级别，确保消息可靠地传递到目标。
- 4) 实时性：支持实时消息传递，使得设备和应用程序能够快速响应事件和状态变化。

MQTT 在物联网中的应用：

- 1) 物联网设备通信：MQTT 可以用于设备之间的通信，实现设备数据的实时传输和远程控制。
- 2) 传感器数据采集：传感器可以使用 MQTT 协议将采集的数据发布到 MQTT 代理服务器，供其它应用程序订阅和处理。
- 3) 消息推送和订阅：应用程序可以使用 MQTT 订阅感兴趣的主体，以接收相关的消息推送。
- 4) 云平台集成：MQTT 可以用于将设备数据集成到云平台中，实现大规模的数据传输和处理。

1.2 MQTT 基本概念

在本节中，将介绍 MQTT 的基本概念，包括发布/订阅模式、主题（Topic）、客户端（Client）和代理服务器（Broker），帮助理解 MQTT 的工作原理和关键组成部分。

发布/订阅模式：

- 1) MQTT 使用发布/订阅模式进行消息传递。
- 2) 发布者（Publisher）将消息发布到特定的主题（Topic）上。
- 3) 订阅者（Subscriber）订阅感兴趣的主体，以接收与该主题相关的消息。
- 4) 发布者和订阅者之间不存在直接连接，而是通过代理服务器进行消息传递。

主题（Topic）：

- 1) 主题是 MQTT 中用于标识消息内容的逻辑名称。
- 2) 主题是以层次结构组织的，使用斜杠（/）进行分隔，例如“sensors/temperature”。
- 3) 主题可以包含多级子主题，例如“sensors/temperature/living-room”。
- 4) 订阅者可以订阅特定的主题，以接收与该主题相关的消息。

客户端（Client）：

- 1) MQTT 客户端是连接到 MQTT 代理服务器的应用程序或设备。
- 2) 客户端可以是发布者、订阅者或同时兼具两者的角色。
- 3) 每个客户端都有一个唯一的客户端标识符（Client Identifier），用于在 MQTT 网络中唯一标识该客户端。

代理服务器（Broker）：

- 1) MQTT 代理服务器是负责接收、路由和传递消息的中间件。
- 2) 代理服务器是 MQTT 网络的核心组件，实现了发布/订阅模式的消息传递机制。

- 3) 代理服务器负责管理客户端的连接、认证和消息传递的可靠性。

MQTT 控制报文:

- 1) MQTT 使用控制报文进行客户端和代理服务器之间的通信。
- 2) 常见的 MQTT 控制报文包括连接请求、连接确认、发布消息、订阅主题、取消订阅等。
- 3) 这些报文用于建立连接、传递消息和管理订阅关系。

1.3 MQTT 客户端

在本章中，将探讨 MQTT 客户端的概念和功能，以及常见的 MQTT 客户端库和框架。

客户端类型和功能:

- 1) MQTT 客户端可以分为设备端和应用程序端。
- 2) 设备端客户端通常是嵌入式设备、传感器或物联网终端，用于发布设备数据或接收远程指令。
- 3) 应用程序端客户端是在应用程序中使用的 MQTT 客户端，用于订阅主题、处理消息和与设备通信。

MQTT 客户端库和框架:

- 1) MQTT 客户端库是为不同编程语言提供的 MQTT 客户端实现。
- 2) 常见的 MQTT 客户端库包括 Paho MQTT（支持多种编程语言）、Eclipse Mosquitto、MQTT.js 等。
- 3) 诸如此类的客户端库提供了简化 MQTT 通信的接口和功能，使开发者能够快速集成 MQTT 功能到应用程序或设备中。

1.4 连接和身份验证

在本章中，将讨论如何与 MQTT 代理服务器建立连接，并介绍 MQTT 客户端身份验证的机制。

连接到 MQTT 代理服务器:

- 1) MQTT 客户端需要通过指定代理服务器的地址和端口来建立连接。
- 2) 客户端可以选择使用安全连接（如 TLS/SSL）或非安全连接（明文）来与代理服务器通信。
- 3) 客户端还可以设置其它连接参数，如超时时间、保持活动时间等。

客户端身份验证机制:

- 1) MQTT 提供了多种身份验证机制，以确保只有授权的客户端能够连接到代理服务器。

- 2) 常见的身份验证方法包括用户名和密码认证、TLS/SSL 证书认证和令牌认证。
- 3) 客户端需要根据代理服务器的配置提供相应的身份验证凭据。

1.5 主题订阅和消息发布

订阅主题:

- 1) 客户端可以通过订阅特定的主题来接收与该主题相关的消息。
- 2) 订阅可以使用通配符来匹配多个主题，如单层通配符（+）和多层通配符（#）。

接收和处理订阅的消息:

- 1) 客户端在订阅的主题上接收到消息时，可以通过回调函数或事件处理程序来处理消息。
- 2) 客户端可以解析和提取消息中的数据，并根据需要执行相应的操作。

发布消息到指定主题:

- 1) 客户端可以使用 MQTT 协议发布消息到指定的主题。
- 2) 发布消息需要指定发布的主题和消息内容。
- 3) 客户端可以选择设置消息的质量等级（QoS），以确保消息的可靠传递。

QoS（消息传递质量）:

- 1) MQTT 定义了三个消息传递质量等级：QoS 0、QoS 1 和 QoS 2。
- 2) QoS 0: 最多一次传递，消息发布者发送消息后不会收到确认或保证消息的可靠传递。
- 3) QoS 1: 至少一次传递，消息发布者会收到消息传递的确认，但可能会出现重复传递。
- 4) QoS 2: 只有一次传递，消息发布者会收到消息传递的确认，并确保消息只被传递一次。

1.6 断开连接和持久化会话

在本章中，将讨论如何断开 MQTT 客户端与代理服务器的连接，并介绍持久化会话的概念。

断开连接:

- 1) 客户端可以显式地断开与代理服务器的连接，以释放资源或结束通信。
- 2) 断开连接后，客户端将无法接收或发送任何消息。

持久化会话:

- 1) MQTT 支持持久化会话，以确保客户端在断开连接后仍然可以接收未读消息。

- 2) 持久化会话可以在客户端重新连接时恢复之前的订阅关系和消息状态。
- 3) 客户端可以选择启用或禁用持久化会话。

1.7 MQTT 与物联网应用的集成

应用集成:

- 1) 远程监控和控制: MQTT 可以用于远程监控和控制物联网设备, 如传感器数据的实时监测和远程设备的控制指令。
- 2) 数据采集和分发: MQTT 可以用于采集分布式传感器数据, 并将数据发送到指定的数据中心或应用程序。
- 3) 事件驱动和通知: MQTT 可以用于实现事件驱动的系统, 其中设备或应用程序通过发布事件通知其它相关方。
- 4) 云平台集成: MQTT 可以与云平台集成, 将设备数据传输到云端进行存储、处理和分析。

第 2 章 MQTT 客户端库使用

在本章中，将介绍如何使用 MQTT 客户端库来开发和实现 MQTT 功能。以 Mosquitto 客户端库为例，展示 MQTT 客户端的基本用法。

2.1 Mosquitto 简介

Mosquitto 是一个流行的 MQTT 代理和客户端库，提供 C 语言的实现。它具有跨平台支持，并支持在 ARM 架构上进行编译和使用。Mosquitto 提供了丰富的功能和配置选项，并有广泛的社区支持。

2.2 Mosquitto 的安装

在我司的 SDK 资源包中带有 MQTT 的源码包，可以自行配置开启编译 MQTT，或者手动编译 MQTT 并将测试程序拷贝到开发板中进行测试，具体请参考第 3 章 MQTT 环境搭建。如果镜像中的固件已经搭建好了 MQTT 环境，可以直接参考 2.3 章节进行测试。

2.3 测试用例

Mosquitto 自带了一个测试程序 `mosquitto_pub` 和 `mosquitto_sub`，用于测试 Mosquitto 代理的发布和订阅功能。这些程序可用于验证 Mosquitto 的基本功能和与其它 MQTT 客户端的通信。

以下是 `mosquitto_pub` 和 `mosquitto_sub` 的使用方法说明：

`mosquitto_pub`：用于发布消息到 Mosquitto 代理。

用法: `mosquitto_pub [选项] -t <主题> -m <消息内容>`

示例: `mosquitto_pub -t mytopic -m "Hello, MQTT"`

选项:

`-h <主机>`: 指定 Mosquitto 代理的主机地址,默认是 localhost。

`-p <端口>`: 指定 Mosquitto 代理的端口,默认为 1883。

`-t <主题>`: 指定要发布的消息的主题。

`-m <消息内容>`: 指定要发布的消息内容。

`mosquitto_sub`: 用于从 Mosquitto 代理订阅消息。

用法: `mosquitto_sub [选项] -t <主题>`

示例: `mosquitto_sub -t mytopic`

选项:

`-h <主机>`: 指定 Mosquitto 代理的主机地址。

`-p <端口>`: 指定 Mosquitto 代理的端口,默认为 1883。

`-t <主题>`: 指定要订阅的消息的主题。

通过运行 `mosquitto_pub` 发布消息,再通过运行 `mosquitto_sub` 订阅消息,用户可以测试 Mosquitto 代理的消息传递功能。确保两个程序运行在相同的网络环境下,并正确指定 Mosquitto 代理的主机地址和端口。

用户可以通过运行 `mosquitto_pub -h` 和 `mosquitto_sub -h` 查看更多选项和使用说明,以下是使用 `mosquitto_pub` 和 `mosquitto_sub` 的一些示例:

执行如下命令订阅消息,这将在本地主机上的 Mosquitto 代理上订阅名为“mytopic”的主题,并开始接收该主题下的消息。

```
mosquitto_sub -h localhost -p 1883 -t mytopic
```

执行如下命令发布消息,这将在本地主机上的 Mosquitto 代理上发布一条消息到名为“mytopic”的主题,并将消息内容设置为“Hello,MQTT”。

```
mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
```

测试结果如图 2.3-1 测试结果。

```
root@hzhy:~# mosquitto_sub -h localhost -p 1883 -t mytopic
Hello, MQTT
Hello, MQTT
Hello, MQTT
Hello, MQTT
Hello, MQTT
Hello, MQTT
Hello, MQTT
Hello, MQTT
^Croot@hzhy:~#
```

```
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~# mosquitto_pub -h localhost -p 1883 -t mytopic -m "Hello, MQTT"
root@hzhy:~#
```

图 2.3-1 测试结果

第 3 章 MQTT 环境搭建

搭建 MQTT Broker 可以根据根文件系统的不同，选择不同的方式进行搭建。如果根文件系统是 ubuntu，可参考 3.1 章进行搭建，ubuntu 搭建 MQTT 环境比较简单，仅通过联网下载即可搭建完成；若根文件系统是 buildroot，可参考 3.2 章进行搭建，buildroot 搭建 MQTT 环境需要通过选配工具包来实现；源码编译比较复杂，建议在没有其他简易搭建 MQTT 环境的情况下选用，可参考 3.3 章。

3.1 ubuntu 搭建

准备工作：给设备联网。

接下来依次执行下列命令，安装 MQTT broker 和 mosquitto 客户端。

为确保网络连接正常，请先使用 ping 命令测试外网连通性。

```
ping baidu.com
```

```
root@hzhy:~# ping baidu.com
PING baidu.com (39.156.66.10) 56(84) bytes of data.
64 bytes from 39.156.66.10 (39.156.66.10): icmp_seq=1 ttl=46 time=46.5 ms
64 bytes from 39.156.66.10 (39.156.66.10): icmp_seq=2 ttl=46 time=117 ms
64 bytes from 39.156.66.10 (39.156.66.10): icmp_seq=3 ttl=46 time=200 ms
64 bytes from 39.156.66.10 (39.156.66.10): icmp_seq=4 ttl=46 time=46.4 ms
```

图 3.1-1 测试网络连通性

```
sudo apt-get update
```



```
root@hzhy:~# sudo apt-get update
sudo: unable to resolve host hzhy: Name or service not known
Get:1 https://mirrors.aliyun.com/ubuntu-ports focal InRelease [265 kB]
Get:2 https://mirrors.aliyun.com/ubuntu-ports focal-security InRelease [128 kB]
Get:3 https://mirrors.aliyun.com/ubuntu-ports focal-updates InRelease [128 kB]
Get:4 https://mirrors.aliyun.com/ubuntu-ports focal-backports InRelease [128 kB]
Get:5 https://mirrors.aliyun.com/ubuntu-ports focal-security/restricted Sources [58.4 kB]
Get:6 https://mirrors.aliyun.com/ubuntu-ports focal-security/universe Sources [258 kB]
Get:7 https://mirrors.aliyun.com/ubuntu-ports focal-security/multiverse Sources [21.9 kB]
Get:8 https://mirrors.aliyun.com/ubuntu-ports focal-security/main Sources [335 kB]
Get:9 https://mirrors.aliyun.com/ubuntu-ports focal-security/main arm64 Packages [2736 kB]
Get:10 https://mirrors.aliyun.com/ubuntu-ports focal-security/main Translation-en [514 kB]
Get:11 https://mirrors.aliyun.com/ubuntu-ports focal-security/restricted arm64 Packages [59.0 kB]
Get:12 https://mirrors.aliyun.com/ubuntu-ports focal-security/restricted Translation-en [518 kB]
Get:13 https://mirrors.aliyun.com/ubuntu-ports focal-security/universe arm64 Packages [965 kB]
Get:14 https://mirrors.aliyun.com/ubuntu-ports focal-security/universe Translation-en [221 kB]
Get:15 https://mirrors.aliyun.com/ubuntu-ports focal-security/multiverse arm64 Packages [7244 B]
Get:16 https://mirrors.aliyun.com/ubuntu-ports focal-updates/multiverse Sources [33.7 kB]
Get:17 https://mirrors.aliyun.com/ubuntu-ports focal-updates/restricted Sources [59.6 kB]
Get:18 https://mirrors.aliyun.com/ubuntu-ports focal-updates/universe Sources [415 kB]
Get:19 https://mirrors.aliyun.com/ubuntu-ports focal-updates/main Sources [634 kB]
Get:20 https://mirrors.aliyun.com/ubuntu-ports focal-updates/main arm64 Packages [3053 kB]
Get:21 https://mirrors.aliyun.com/ubuntu-ports focal-updates/main Translation-en [596 kB]
Get:22 https://mirrors.aliyun.com/ubuntu-ports focal-updates/restricted arm64 Packages [63.5 kB]
Get:23 https://mirrors.aliyun.com/ubuntu-ports focal-updates/restricted Translation-en [540 kB]
Get:24 https://mirrors.aliyun.com/ubuntu-ports focal-updates/universe arm64 Packages [1187 kB]
Get:25 https://mirrors.aliyun.com/ubuntu-ports focal-updates/universe Translation-en [383 kB]
Get:26 https://mirrors.aliyun.com/ubuntu-ports focal-updates/multiverse arm64 Packages [13.1 kB]
Fetched 13.2 MB in 8s (1725 kB/s)
Reading package lists... Done
```

图 3.1-2 更新软件源信息

```
sudo apt-get install mosquitto
```

```
root@hzhy:~# sudo apt-get install mosquitto
sudo: unable to resolve host hzhy: Name or service not known
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libdlt2 libev4 libuv1 libwebsockets15
Suggested packages:
  apparmor
The following NEW packages will be installed:
  libdlt2 libev4 libuv1 libwebsockets15 mosquitto
0 upgraded, 5 newly installed, 0 to remove and 39 not upgraded.
Need to get 444 kB of archives.
After this operation, 1332 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 https://mirrors.aliyun.com/ubuntu-ports focal-security/main arm64 libuv1 arm64 1.34.2-1ubuntu1.5 [75.0 kB]
Get:2 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 libdlt2 arm64 2.18.4-0.1 [48.2 kB]
Get:3 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 libev4 arm64 1:4.31-1 [29.3 kB]
Get:4 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 libwebsockets15 arm64 3.2.1-3 [142 kB]
Get:5 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 mosquitto arm64 1.6.9-1 [149 kB]
Fetched 444 kB in 2s (250 kB/s)
```

图 3.1-3 安装 MQTT broker

```
sudo apt-get install mosquitto-clients
```

```

root@hzhy:~# sudo apt-get install mosquitto-clients
sudo: unable to resolve host hzhy: Name or service not known
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  libmosquitto1
The following NEW packages will be installed:
  libmosquitto1 mosquitto-clients
0 upgraded, 2 newly installed, 0 to remove and 39 not upgraded.
Need to get 99.4 kB of archives.
After this operation, 418 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 libmosquitto1 arm64 1.6.9-1 [43.3 kB]
Get:2 https://mirrors.aliyun.com/ubuntu-ports focal/universe arm64 mosquitto-clients arm64 1.6.9-1 [56.2 kB]
Fetched 99.4 kB in 1s (110 kB/s)

```

图 3.1-4 安装 mosquitto 客户端

3.2 buildroot 搭建

3.2.1 配置选择

- 1) 在 SDK 的 buildroot 顶层目录执行命令选配 mosquitto 包，执行如下命令打开配置界面：

```
make menuconfig
```

- 2) 按下键选中 “Target packages”，按下 enter 键；
- 3) 按下键选中 “Networking applications”，按下 enter 键；
- 4) 按下键选中 “mosquitto”，按下空格键选择，选择之后 mosquitto 前面的 ☐ 中会出现 “*” 号，在选择 mosquitto 的时候默认会选择 install the mosquitto broker，不必更改；
- 5) 按右键选择 save，在弹出的窗口中选中 “OK”，按下 enter 键；
- 6) 在弹出的窗口中，按下 enter 键退出保存界面；
- 7) 然后按右键选择 exit，按下 enter 键直到退出配置选择界面。


```

Buildroot -gc9c98d7-dirty Configuration
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty submenus ---). Highlighted letters are hotkeys. Pressing <Y> selects a
feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is
excluded

Target options --->
Build options --->
Toolchain --->
System configuration --->
Kernel --->
Target packages --->
Filesystem images --->
Bootloaders --->
Host utilities --->
Legacy config options --->

<select> < Exit > < Help > < Save > < Load >
    
```

```

Target packages
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty submenus ---). Highlighted letters are hotkeys. Pressing <Y> selects a
feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is
excluded

[*] Show packages that are also provided by busybox
[ ] Individual binaries
[ ] Install the watchdog daemon startup script
Audio and video applications --->
Compressors and decompressors --->
Debugging, profiling and benchmark --->
Development tools --->
Filesystem and flash utilities --->
Fonts, cursors, icons, sounds and themes --->
Games --->
Graphic libraries and applications (graphic/text) --->
Hardware handling --->
Interpreter languages and scripting --->
Libraries --->
Mail --->
Miscellaneous --->
[*] Networking applications --->
Package managers --->
Real-Time --->
Security --->
Shell and utilities --->
System tools --->
Text editors and viewers --->

<select> < Exit > < Help > < Save > < Load >
    
```

```

Networking applications
Arrow keys navigate the menu. <Enter> selects submenus --- (or empty submenus ---). Highlighted letters are hotkeys. Pressing <Y> selects a
feature, while <N> excludes a feature. Press <Esc><Esc> to exit, <?> for Help, </> for Search. Legend: [*] feature is selected [ ] feature is
excluded

[*] minissdpd
[ ] mpg-streamer
[ ] modem-manager
[ ] mongrel2
[ ] mosh
[*] m_squitto
[*] install the mosquitto broker
[ ] mROUTED
*** mrp needs a toolchain w/ threads, kernel headers >= 5.0 ***
[ ] mstpd
[ ] mtr
[ ] nbd
[ ] netftp
[ ] ndisc6 tools
[ ] net-tools
[ ] netatalk
[ ] netcalc
[ ] netcat
[ ] netcat-openbsd
[ ] netplug
[ ] netsnmp
[ ] netstat-nat
*** NetworkManager needs udev /dev management and a glibc toolchain w/ headers >= 4.6, dynamic library, wc

<select> < Exit > < Help > < Save > < Load >
    
```

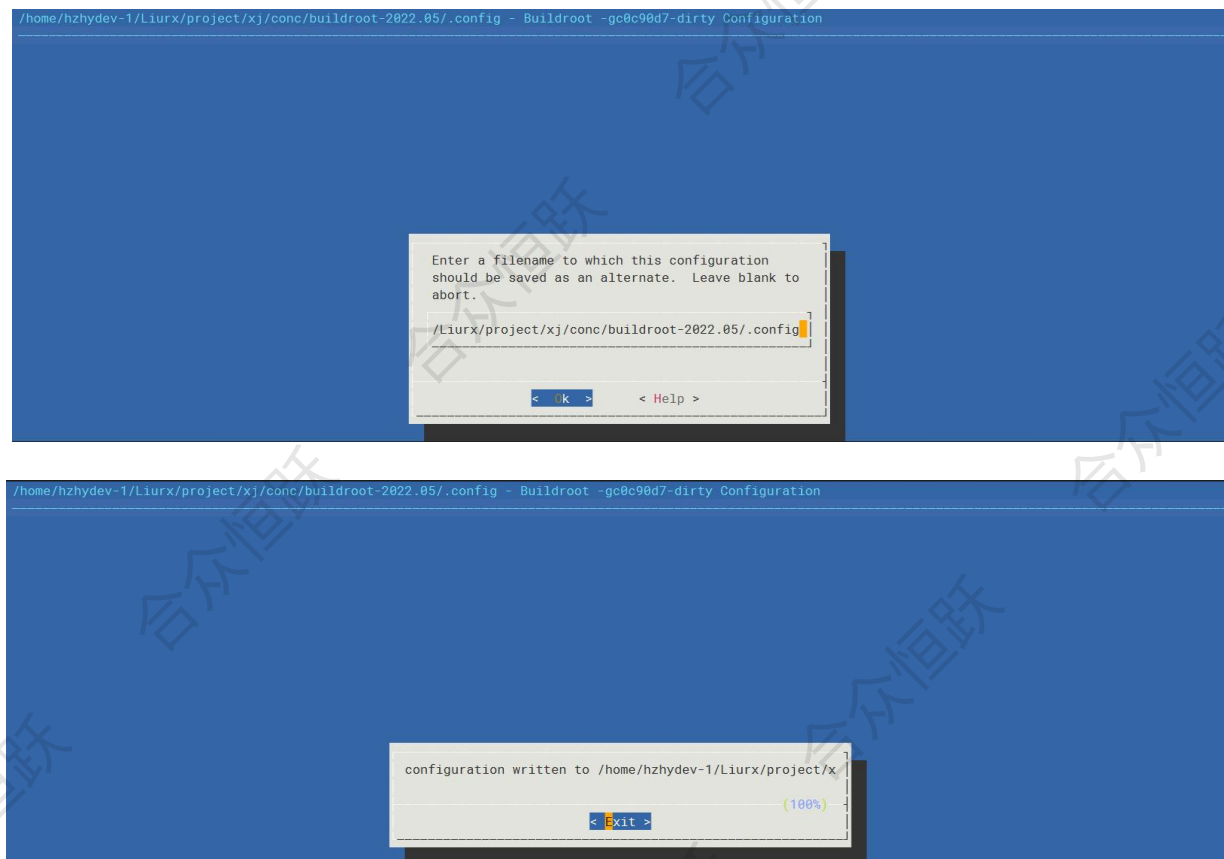


图 3.2-1 buildroot 安装选配 mosquitto 界面

3.2.2 编译及固化

根据开发板的 SDK 使用手册，进行整体编译或者单独编译 rootfs，编译完成后根据烧录手册更新固件，固件更新之后 mosquitto 已移植至开发板。

```
root@hzhyxa-and1:/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05# make
>>> Finalizing host directory
>>> Finalizing target directory
mkdir -p /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/etc/meson
sed -e "s%@TARGET_CFLAGS%'-D_LARGEFILE_SOURCE', '-D_LARGEFILE64_SOURCE', '-D_FILE_OFFSET_BITS=64', '-O3', '-g0', '-D_FORTIFY_SOURCE=1'@PKG_TARGET_CFLAGS%g" -e "s%@TARGET_LD_FLAGS%'-D_LARGEFILE_SOURCE', '-D_LARGEFILE64_SOURCE', '-D_FILE_OFFSET_BITS=64', '-O3', '-g0', '-D_FORTIFY_SOURCE=1'@PKG_TARGET_CXXFLAGS%g" -e "s%@TARGET_CC%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-gcc-ar'g" -e "s%@TARGET_AR%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-strip'g" -e "s%@TARGET_STRIP%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-strip'g" -e "s%@TARGET_ARCH%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/cmake'g" -e "s%@PKGCONF_HOST_BINARY%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/pkgconf'g" -e "s%@HOST_DIR%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/arm-buildroot-linux-gnueabi/sysroot'g" -e "s%@STATIC%false%g" /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/support/misc/cross-compilation.conf.in > /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/etc/meson/cross-compilation.conf.in
sed -e "s%@TARGET_CC%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-gcc-ar'g" -e "s%@TARGET_CXX%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-strip'g" -e "s%@TARGET_STRIP%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/arm-linux-gnueabi-strip'g" -e "s%@TARGET_ARCH%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/cmake'g" -e "s%@PKGCONF_HOST_BINARY%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/bin/pkgconf'g" -e "s%@HOST_DIR%'/home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/arm-buildroot-linux-gnueabi/sysroot'g" -e "s%@STATIC%false%g" /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/support/misc/cross-compilation.conf.in > /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/host/etc/meson/cross-compilation.conf.in
if grep -q CONFIG_ASH-y /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/build/busybox-1.35.0/.config; then grep -qE '^/bin/ash$' /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/target/etc/shells || echo "/bin/ash" >> /home/hzhydev-1/Liurx/project/xj/conc/buildroot-2022.05/output/target/etc/shells; fi
```

图 3.2-2 编译 buildroot

3.3 源码交叉编译

下载源码，交叉编译 mosquitto，这里以移植版本为 2.0.21 的 mosquitto 为例进行说明，使用的 openssl 版本为 1.1.1g，cJSON 版本为 1.7.17。

该版本的 mosquitto 依赖于 openssl 和 cJSON，首先交叉编译 openssl 和 cJSON，编译 mosquitto 的时候需要链接 openssl 和 cJSON 的库和头文件。

- 1) 首先创建一个工作目录，在这个工作目录中进行操作

```
mkdir libraries
cd libraries
mkdir ssl
mkdir cJSON
mkdir mqtt
```

```
LRX@ubuntu:~$ tree libraries/ -L 1
libraries/
├── cJSON
├── mqtt
└── ssl
```

图 3.3-1 工作目录结构

- 2) 下载并编译 openssl

- 进入刚才创建的 ssl 目录；

```
cd ssl
```

- 下载 openssl 压缩包；

```
wget https://github.com/openssl/openssl/releases/download/OpenSSL_1_1_1g/openssl-1.1.1g.tar.gz
```

- 将 openssl 压缩包解压；

```
tar -xf openssl-1.1.1g.tar.gz
```

- 导入环境变量；

导入环境变量命令如下，开发人员应根据实际 SDK 解压路径导入实际 buildroot 编译链环境变量。

```
export PATH=/home/LRX/tools/toolchains/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin:$PATH
```

- 配置 OpenSSL 的交叉编译环境；

```
./Configure linux-aarch64 \
```

```
no-asm \
shared \
no-async \
--prefix=$(pwd)/ssl_result \
--cross-compile-prefix=aarch64-none-linux-gnu-
```

- 执行下面命令，删除 Makefile 文件的 -m64，执行后，可以避免出现这个编译错误：error: unrecognized command line option '-m64'；

```
sed -i 's/-m64/' Makefile
```

- 编译、安装；

```
make && make install
```

- 编译成功后，在 openssl-OpenSSL_1_1_1g/目录会生成一个 ssl_result 目录，可以看到里面生成的库，参考图 3.3-2 ssl_result 目录结构。

```
LRX@ubuntu:~/libraries/ssl/openssl-1.1.1g$ tree ./ssl_result/ -L 2
./ssl_result/
├── bin
│   ├── c_rehash
│   └── openssl
├── include
│   └── openssl
├── lib
│   ├── engines-1.1
│   ├── libcrypto.a
│   ├── libcrypto.so -> libcrypto.so.1.1
│   ├── libcrypto.so.1.1
│   ├── libssl.a
│   ├── libssl.so -> libssl.so.1.1
│   └── libssl.so.1.1
├── pkgconfig
├── share
│   ├── doc
│   └── man
└── ssl
    ├── certs
    ├── ct_log_list.cnf
    ├── ct_log_list.cnf.dist
    ├── misc
    ├── openssl.cnf
    ├── openssl.cnf.dist
    └── private

14 directories, 12 files
```

图 3.3-2 ssl_result 目录结构

3) 下载并编译 cJSON

- 进入之前创建的 cJSON 目录；

```
cd cJSON
```

- 下载 cJSON 压缩包；

```
wget https://github.com/DaveGamble/cJSON/archive/refs/tags/v1.7.17.tar.gz
```

- 导入环境变量;

导入环境变量命令如下, 开发人员应根据实际 SDK 解压路径导入实际 buildroot 编译链环境变量。

```
export PATH=/home/LRX/tools/toolchains/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin:$PATH
```

- 编译脚本, 将下面的 shell 脚本保存成 compile.sh, 与下载的 v1.7.17.tar.gz 放在同一目录;

注: 需要修改 CROSS_COMPILE_TOOL 字段的变量为真正的编译链前缀

```
#!/bin/sh

export cJSON_VER=1.7.17

export cJSON_DIR_NAME=cJSON-${cJSON_VER}

CROSS_COMPILE_TOOL=aarch64-none-linux-gnu-

# 1.解压缩

rm ${cJSON_DIR_NAME} -rf

tar xzf v${cJSON_VER}.tar.gz || { echo "Error: Failed to extract tar file"; exit 1; }

# 2.编译

cd ${cJSON_DIR_NAME} || { echo "Error: Failed to enter directory ${cJSON_DIR_NAME}"; exit 1; }

# 2.1 修改安装目录路径

sed -i 's/PREFIX ?= /a PREFIX ?= $(PWD)/../${cJSON_DIR_NAME}_result' Makefile

sed -i 's/PREFIX ?= /usr/local/d' Makefile

make CC=${CROSS_COMPILE_TOOL}gcc || { echo "Error: Compilation failed"; exit 1; }

make install || { echo "Error: Installation failed"; exit 1; }

cd ..
```

- 给 compile.sh 加可执行权限;

```
chmod +x compile.sh
```


- 执行 compile.sh 脚本进行编译；

```
./compile.sh
```

- 编译完成后可以在 cJSON 目录下生成 cJSON-1.7.17_result 目录，可以看到里面生成的库，参考图 3.3-3 cJSON-1.7.17_result 目录结构。

```
LRX@ubuntu:~/libraries/cJSON$ tree cJSON-1.7.17_result
cJSON-1.7.17_result
├── include
│   └── cJSON
│       ├── cJSON.h
│       └── cJSON_Utils.h
└── lib
    ├── libcjson.so -> libcjson.so.1
    ├── libcjson.so.1 -> libcjson.so.1.7.17
    ├── libcjson.so.1.7.17
    ├── libcjson_utils.so -> libcjson_utils.so.1
    ├── libcjson_utils.so.1 -> libcjson_utils.so.1.7.17
    └── libcjson_utils.so.1.7.17

4 directories, 8 files
```

图 3.3-3 cJSON-1.7.17_result 目录结构

4) 下载并编译 mosquitto

- 进入之前创建的 mqtt 目录；

```
cd mqtt
```

- 下载 mosquitto 压缩包；

```
wget https://mosquitto.org/files/source/mosquitto-2.0.21.tar.gz
```

- 将 mosquitto 压缩包解压；

```
tar -xf mosquitto-2.0.21.tar.gz
```

- 修改 mosquitto-2.0.21 目录下的 config.mk，请参考图 3.3-4 config.mk 修改截图；

```
#指定 ssl 库、cJSON 库的头文件和库文件
```

```
CFLAGS += -I/home/LRX/libraries/cJSON/cJSON-1.7.17_result/include -I/home/LRX/libraries/ssl/openssl-1.1.1g/ssl_result/include
```

```
LDFLAGS+= -L/home/LRX/libraries/cJSON/cJSON-1.7.17_result/lib -L/home/LRX/libraries/ssl/openssl-1.1.1g/ssl_result/lib -lssl -lcrypto -dl
```

```
#指定安装目录
```

```
DESTDIR=/home/LRX/libraries/mqtt/mosquitto-2.0.21/mosquitto-2.0.21_result
```

```
151 # 指定 ssl 库、cJSON 库的头文件和库文件
152 CFLAGS += -I/home/LRX/libraries/cJSON/cJSON-1.7.17_result/include -I/home/LRX/libraries/ssl/openssl-1.1.1g/ssl_result/include
153 LDFLAGS+= -L/home/LRX/libraries/cJSON/cJSON-1.7.17_result/lib -L/home/LRX/libraries/ssl/openssl-1.1.1g/ssl_result/lib -lssl -lcrypto -dl
154 # 指定安装目录
155 DESTDIR=/home/LRX/libraries/mqtt/mosquitto-2.0.21/mosquitto-2.0.21_result
```

图 3.3-4 config.mk 修改截图

注：

- (1) 修改 CFLAGS 字段中 -I 后的目录为自己的工作目录编译生成的 cJSON 和 openssl 的 lib 目录（使用绝对路径）；
- (2) 修改 LDFLAGS 字段中 -L 后的目录为自己的工作目录编译生成的 cJSON 和 openssl 的 include 目录（使用绝对路径）；
- (3) -lssl -lcrypto -dl 这几个选项也要加，不然链接时可能会报错；
- (4) DESTDIR 目录一定要指定，不然 make install 的时候可能会安装到根目录去。

- 编译、安装：

```
make CC=aarch64-buildroot-linux-gnu-gcc CXX=aarch64-buildroot-linux-gnu-g++ && make install
```

- 编译完成后可以在 mosquitto-2.0.21 下生成 mosquitto-2.0.21_result 目录，可以看到里面生成的库和二进制文件，请参考图 3.3-5 mosquitto-2.0.21_result 目录结构。

```
LRX@ubuntu:~/libraries/mqtt/mosquitto-2.0.21$ tree mosquitto-2.0.21_result -L 4
mosquitto-2.0.21_result
├── etc
│   └── mosquitto
│       ├── aclfile.example
│       ├── mosquitto.conf.example
│       ├── pskfile.example
│       └── pwfile.example
├── usr
│   └── local
│       ├── bin
│       │   ├── mosquitto_ctrl
│       │   ├── mosquitto_passwd
│       │   ├── mosquitto_pub
│       │   ├── mosquitto_rr
│       │   └── mosquitto_sub
│       ├── include
│       │   ├── mosquitto_broker.h
│       │   ├── mosquitto.h
│       │   ├── mosquitto_plugin.h
│       │   ├── mosquitto.h
│       │   └── mqtt_protocol.h
│       ├── lib
│       │   ├── libmosquitto.so -> libmosquitto.so.1
│       │   ├── libmosquitto.so.1
│       │   ├── libmosquitto.so -> libmosquitto.so.1
│       │   ├── libmosquitto.so.1
│       │   ├── mosquitto_dynamic_security.so
│       │   └── pkgconfig
│       ├── sbin
│       │   └── mosquitto
│       ├── share
│       └── man
12 directories, 20 files
```

图 3.3-5 mosquitto-2.0.21_result 目录结构

- 5) 部署到开发板

- 在开发板上创建 mosquitto 配置文件目录；

```
sudo mkdir -p /etc/mosquitto
```

- 在开发板上创建 mosquitto 配置文件：

```
cat > /etc/mosquitto/mosquitto.conf <<EOF
pid_file /var/run/mosquitto.pid
listener 1883 0.0.0.0
user root
allow_anonymous true
log_dest file /var/log/mosquitto.log
EOF
```

- 将 mosquitto-2.0.21_result 拷贝到开发板/root 下，并执行以下命令：

```
cp mosquitto-2.0.21_result/usr/local/sbin/mosquitto* /usr/bin
cp mosquitto-2.0.21_result/usr/local/sbin/*mosquitto*.so* /usr/lib
Cp mosquitto-2.0.21_result/usr/local/sbin/mosquitto /usr/sbin
```

- 运行 mosquitto，-c 是指定 mosquitto 配置文件 mosquitto.conf 的路径。

```
/usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf
```

6) 开机自启动：以下两种方法均可实现开机自启动，请根据系统管理方式选择合适的开启自启动方法

方法一：适用于支持 systemd 的系统，创建 systemd 服务文件，将下面的内容命名为 mosquitto.service，保存在开发板/etc/systemd/system/路径下。

```
[Unit]
Description=Mosquitto MQTT Broker
After=network.target

[Service]
ExecStart=/usr/bin/mosquitto -c /etc/mosquitto/mosquitto.conf
Restart=always

[Install]
WantedBy=multi-user.target
```

方法二：适用于支持 SystemV 的系统，创建 S50mosquitto 脚本，将下面的内容命名为 S50mosquitto

o, 保存在开发板/etc/init.d/路径下。

```
#!/bin/sh

start() {
    printf "Starting mosquitto: "
    start-stop-daemon -S -q -m -b -p /var/run/mosquitto.pid \
        --exec /usr/sbin/mosquitto \
        -- -c /etc/mosquitto/mosquitto.conf
    [ $? = 0 ] && echo "OK" || echo "FAIL"
}

stop() {
    printf "Stopping mosquitto: "
    start-stop-daemon -K -q -p /var/run/mosquitto.pid
    [ $? = 0 ] && echo "OK" || echo "FAIL"
}

restart() {
    stop
    start
}

case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    restart|reload)
        restart
        ;;
    *)
        echo "Usage: $0 {start|stop|restart|reload}"
        exit 1
    esac
```

```
;;  
  
*)  
  
    echo "Usage: $0 {start|stop|restart}"  
  
    exit 1  
  
esac  
  
exit $?
```

3.4 环境测试

可以参照 2.3 章节，使用客户端库去测试环境是否搭建成功。

第 4 章 MQTT 开发案例

4.1 案例说明

该 mqtt_demo 工程实现了一个双向 MQTT 客户端。支持三种运行模式：

- 持续发布模式：定期向指定主题发布测试消息
- 订阅模式：订阅指定主题并接收消息
- 单次发送模式：发送一条指定消息后退出

实现了 MQTT 协议的基本功能（连接、发布、订阅）。

可以通过参数配置代理服务器地址和端口，发布/订阅主题，QoS 等级，发布频率（持续发布模式），直接发送的消息内容（单次发送模式）。

4.2 案例测试

下面将使用开发板的 mosquitto 代理服务器对该 mqtt_demo 工程进行测试，测试步骤如下：

1) 首先确保开发板的 mosquitto 代理服务器正常运行，如图 4.2-1 mosquitto 代理服务器正常运行所示，能 grep 到 mosquitto 进行之后再执行步骤“2)”，如果不能 grep 到，则需根据系统管理方式启动 mosquitto 代理服务器。

对于 SystemV 管理的系统执行/etc/init.d/S50mosquitto start 命令

对于 systemd 管理的系统执行 systemctl start mosquitto 命令

```
root@hzhy:~# ps -e | grep "mos"
621 ?        00:00:00 mosquitto
```

图 4.2-1 mosquitto 代理服务器正常运行

2) 启动开发板，将 mqtt_demo 目录下的 mqtt_dual_client 可执行文件拷贝至开发板文件系统的/root 目录下，登录串口终端执行如下命令查看程序参数说明。

```
chmod +x mqtt_dual_client
./mqtt_dual_client --help
```

```
root@hzhy:~# ./mqtt_dual_client --help

===== HZHY MQTT双向通信客户端 v2.0 =====
用法: ./mqtt_dual_client [选项]

选项:
-h, --host <地址>      MQTT代理服务器地址 (默认: localhost)
-p, --port <端口>      MQTT代理端口 (默认: 1883)
-P, --pub-topic <主题> 发布消息的主题
-S, --sub-topic <主题> 订阅消息的主题
-i, --id <客户端ID>    客户端标识符 (默认: MqttDualClient_<PID>)
-q, --qos <等级>       服务质量等级 (0-2, 默认: 1)
-f, --freq <频率>      消息发布频率(Hz) (仅持续发布模式有效)
-m, --message <消息>   直接发送的消息内容 (单次发送模式)
-v, --version           显示版本信息
--help                 显示此帮助信息

模式说明:
1. 持续发布模式: 指定-P和-f参数
2. 订阅模式: 指定-S参数
3. 单次发送模式: 指定-P和-m参数

示例:
# 持续发布测试消息
./mqtt_dual_client -h broker -P sensor -f 1

# 单次发送控制命令
./mqtt_dual_client -h broker -P control -m "{cmd:reset}"

# 订阅日志主题
./mqtt_dual_client -h broker -S logs
```

图 4.2-2 程序使用帮助说明

- 3) 在开发板的串口终端执行如下命令。

```
./mqtt_dual_client -S topic/test
```

```
root@hzhy:~# ./mqtt_dual_client -S topic/test
[开始订阅] 等待消息 ...
[连接成功] localhost:1883
[消息接收] topic/test: www.hzhytech.com (QoS:1)
```

图 4.2-3 从 mosquitto 代理服务器接收消息

- 4) 在开发板的 ssh 终端执行如下命令。

```
./mqtt_dual_client -P topic/test -m "www.hzhytech.com"
```

```
root@hzhy:~# ./mqtt_dual_client -P topic/test -m "www.hzhytech.com"
[单次发送] 主题: topic/test
消息: www.hzhytech.com
[连接成功] localhost:1883
[连接断开]
[资源释放] 完成
```

图 4.2-4 向 mosquitto 代理服务器发布消息

注: 订阅和发布主题一致的情况下, 订阅客户端才能从mosquitto代理服务器收到消息。

4.3 案例编译

- 1) 导入环境变量。导入环境变量命令如下, 开发人员应根据实际 SDK 解压路径导入实际 buildroot 编译链环境变量


```
export PATH=$PATH:/home/LRX/SDK_NAME/buildroot_toolchain_path/bin/
```

2) 编译

```
make
```

4.4 关键代码分析

1) 创建 MQTT 实例

```
// 创建MQTT客户端实例
mqttClient = mosquitto_new(clientConfig->clientId, true, clientConfig);
if (!mqttClient) {
    fprintf(stderr, "[错误] 无法创建MQTT客户端实例\n");
    return false;
}
```

图 4.4-1 创建 MQTT 实例代码片段

2) 设置回调函数

```
// 设置回调函数
mosquitto_connect_callback_set(mqttClient, HzMqttConnect);
mosquitto_disconnect_callback_set(mqttClient, HzMqttDisconnect);
mosquitto_message_callback_set(mqttClient, HzMqttMessage);
mosquitto_publish_callback_set(mqttClient, HzMqttPublish);
```

图 4.4-2 设置回调函数代码片段

3) 连接 MQTT 代理服务器

```
// 连接到MQTT代理
ret = mosquitto_connect(mqttClient, clientConfig->brokerHost, clientConfig->brokerPort, DEFAULT_KEEPAIVE_SEC);
if (ret != MOSQ_ERR_SUCCESS) {
    fprintf(stderr, "[错误] 连接请求失败: %s\n", mosquitto_strerror(ret));
    return false;
}
```

图 4.4-3 连接 MQTT 代理服务器代码片段

4) 发布消息

```

void RunPublishLoop(MqttClientConfig* cfg) {
    const int intervalUs = 1000000 / cfg->publishIntervalHz;
    int count = 0;
    char buf[MAX_MESSAGE_LENGTH];

    printf("[开始发布] 主题: %s, 频率: %dHz\n", cfg->publishTopic, cfg->publishIntervalHz);

    while (isRunning) {
        snprintf(buf, sizeof(buf), "测试消息 #d", ++count);
        int ret = mosquitto_publish(mqttClient, NULL, cfg->publishTopic,
                                    strlen(buf), buf, cfg->qosLevel, false);
        if (ret != MOSQ_ERR_SUCCESS) {
            fprintf(stderr, "[发布失败] %s\n", mosquitto_strerror(ret));
        }
        usleep(intervalUs);
    }
}

void SendSingleMessage(MqttClientConfig* cfg) {
    printf("[单次发送] 主题: %s\n消息: %s\n", cfg->publishTopic, cfg->message);

    int ret = mosquitto_publish(mqttClient, NULL, cfg->publishTopic,
                                strlen(cfg->message), cfg->message,
                                cfg->qosLevel, false);
    if (ret != MOSQ_ERR_SUCCESS) {
        fprintf(stderr, "[发送失败] %s\n", mosquitto_strerror(ret));
        isRunning = false;
    }
}

```

图 4.4-4 发布消息代码片段

5) 订阅主题

```

void HzMqttConnect(struct mosquitto* mosq, void* userData, int resultCode) {
    MqttClientConfig* clientConfig = (MqttClientConfig*)userData;

    if (resultCode == MOSQ_ERR_SUCCESS) {
        printf("[状态] 成功连接到MQTT代理 %s:%d\n", clientConfig->brokerHost, clientConfig->brokerPort);

        // 订阅主题
        if (strlen(clientConfig->subscribeTopic) > 0) {
            int ret = mosquitto_subscribe(mosq, NULL, clientConfig->subscribeTopic, clientConfig->qosLevel);
            if (ret != MOSQ_ERR_SUCCESS) {
                fprintf(stderr, "[错误] 订阅主题失败: %s\n", mosquitto_strerror(ret));
            } else {
                printf("[状态] 已订阅主题: %s (QoS: %d)\n", clientConfig->subscribeTopic, clientConfig->qosLevel);
            }
        }
    } else {
        fprintf(stderr, "[错误] 连接失败: %s\n", mosquitto_strerror(resultCode));
        isRunning = false;
    }
}

```

图 4.4-5 订阅主题代码片段

第 5 章 常用接口说明

如下列出了 mosquitto 中常用的 API 接口说明。

- 1) 初始化 Mosquitto 库
`mosquitto_lib_init()`
- 2) 创建 MQTT 客户端实例
`mosq = mosquitto_new(client_id, clean_session, userdata)`
- 3) 设置连接回调函数
`mosquitto_connect_callback_set(mosq, on_connect)`
- 4) 设置断开连接回调函数
`mosquitto_disconnect_callback_set(mosq, on_disconnect)`
- 5) 设置消息接收回调函数
`mosquitto_message_callback_set(mosq, on_message)`
- 6) 连接到 MQTT 代理
`mosquitto_connect(mosq, host, port, keepalive)`
- 7) 循环处理网络事件
`mosquitto_loop_start(mosq)`
- 8) 订阅主题
`mosquitto_subscribe(mosq, NULL, topic, qos)`
- 9) 发布消息
`mosquitto_publish(mosq, NULL, topic, payload, payloadlen, qos, retain)`
- 10) 等待消息接收
`mosquitto_loop_stop(mosq, true)`
- 11) 断开连接并清理资源

```
mosquitto_disconnect(mosq)
```

```
mosquitto_destroy(mosq)
```

12) 清理 Mosquitto 库

```
mosquitto_lib_cleanup()
```

更多接口的使用请参考 `mosquitto` 库的 API 说明文档以及其示例。